15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

10 Oct 2026
Day 1/15

Day 1: Big O & Time/Space Complexity

1. What is Big O?

- Big O notation tells us how the running time or memory usage of an algorithm grows as the input size (n) increases.
- It gives us the upper bound (worst case) of the algorithm's performance.
- It helps us compare algorithms and choose the most efficient one.

2. Common Time Complexities

Complexity	Name	Growth Rate	Example
$O(1)$	Constant	No change	Accessing array element
$O(\log n)$	Logarithmic	Slow growth	Binary search
$O(n)$	Linear	Linear growth	Loop through array
$O(n \log n)$	Linearithmic	Moderate	Merge sort, Quick sort
$O(n^2)$	Quadratic	Fast growth	Nested loops
$O(2^n)$	Exponential	Very fast	Recursive Fibonacci
$O(n!)$	Factorial	Extremely fast	Permutations

3. Common Space Complexities

- $O(1)$ – Constant space (no extra space needed)
- $O(n)$ – Linear space (e.g. array, list)
- $O(n^2)$ – 2D array / matrix
- $O(2^n)$ – Recursion stack (e.g. subset generation)

4. Example – Time Complexity

1. Loop through an array ($O(n)$)

```
int SumArray(int[] arr)
{
    int sum = 0;
    for (int i = 0; i < arr.Length; i++)
    {
        sum += arr[i];
    }
    return sum;
}
```

The loop runs n times, so time complexity is $O(n)$.

2. Nested loop ($O(n^2)$)

```
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        Console.WriteLine(i + " " + j);
    }
}
```

Inner loop runs n times for each outer loop, so total is $O(n^2)$.

5. Example – Space Complexity

1. Using extra array ($O(n)$)

```
int[] GetSquares(int[] arr)
{
    int[] result = new int[arr.Length];
    for (int i = 0; i < arr.Length; i++)
    {
        result[i] = arr[i] * arr[i];
    }
    return result;
}
```

We are using an extra array of size n, so space complexity is $O(n)$.

6. Real Life Analogy

- $O(1)$ – Picking a book from a shelf (direct access)
- $O(\log n)$ – Finding a word in a dictionary (binary search)
- $O(n)$ – Checking each student in a class
- $O(n^2)$ – Comparing every student with every other
- $O(2^n)$ – Trying all combinations
- $O(n!)$ – Arranging n people in a line

7. Tips to Remember

- Lower complexity is better.
- In most interviews, $O(n)$ and $O(\log n)$ are most common.
- Look for loops, nested loops, recursion and extra space while analyzing complexity.
- Always consider the worst-case scenario.

8. Practice Question

Q1. What is the time and space complexity of the following code?

```
int Count(int[] arr)
{
    int count = 0;
    foreach (var num in arr)
    {
        if (num % 2 == 0)
            count++;
    }
    return count;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Q2. What is the time complexity of binary search?

A) $O(n)$ B) $O(\log n)$ C) $O(n^2)$ D) $O(1)$

9. Today's Takeaway

- ☐ Understand what Big O means.
- ☐ Learn common time & space complexities.
- ☐ Analyze code and identify the complexity.
- ☐ Practice a few questions daily.

Keep Going ☺

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

11 Oct 2026
Day 2/15

Day 2 : Arrays

1. What is an Array?

- An array is a collection of elements of the same data type stored in a contiguous block of memory.
- It allows us to store multiple values under a single variable name.
- We can access elements using an index (starting from 0 in C#).

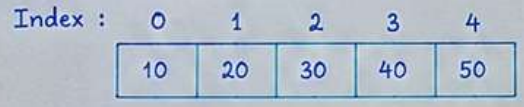
2. Key Characteristics

- Fixed size (cannot be changed after creation)
- Homogeneous data type (all elements are same type)
- Zero-based indexing (0 to n-1)
- Fast access (O(1) time to access an element)
- Can be single-dimensional or multi-dimensional.

3. Types of Arrays in C#

- 1D Array : e.g. `int[] numbers = new int[5];`
- 2D Array : e.g. `int[,] matrix = new int[3,3];`
- Jagged Array : e.g. `int[][] jagged = new int[3][];`

4. 1D Array Example




```
int[] numbers = { 10, 20, 30, 40, 50 };
```

5. C# Code Examples

(a) Traversing an Array (O(n))

```
int[] numbers = { 10, 20, 30, 40, 50 };
for (int i = 0; i < numbers.Length; i++)
{
    Console.WriteLine(numbers[i]);
}
```

- Visits each element once → O(n) time.

(b) Finding the Maximum Element (O(n))

```
int[] numbers = { 10, 20, 30, 40, 50 };
int max = numbers[0];
for (int i = 1; i < numbers.Length; i++)
{
    if (numbers[i] > max)
        max = numbers[i];
}
```

- Compares each element → O(n) time, O(1) space.

6. 2D Array Example

```
int[,] matrix = new int[2,3]
{
    { 1, 2, 3 },
    { 4, 5, 6 }
};
for (int i = 0; i < 2; i++)
{
    for (int j = 0; j < 3; i++)
    {
        Console.Write(matrix[i, j] + " ");
    }
    Console.WriteLine();
}
```

2D array stores data in rows and columns.

7. Jagged Array Example

```
int[][] jagged = new int[3][];
jagged[0] = new int[] { 1, 2, 3 };
jagged[1] = new int[] { 4, 5 };
jagged[2] = new int[] { 6, 7, 8, 9 };
```

Each row can have a different number of elements.

8. Common Operations

- Access element : `arr[index]`
- Update element : `arr[index] = value`
- Get length : `arr.Length`
- Looping : for / foreach
- Copying : `Array.Copy()` or `Clone()`

9. Real Life Analogies

- Seating arrangement in a row (1D array)
- Spreadsheet (2D array)
- Multiple phone numbers for a contact (jagged array)

10. Tips to Remember

- Index starts from 0 in C#.
- Accessing an invalid index throws IndexOutOfRangeException.
- Arrays are reference types in C# (except for some cases like stackalloc).
- Use List<T> if you need a dynamic size collection.

11. Practice Questions

Q1. Write a C# program to print the sum of all elements in an array.

Q2. Find the largest element in an array.

Q3. Print the elements of a 2D array.

Q4. What is the time complexity of accessing an element in an array?

Keep Going ☺

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

12 Oct 2026
Day 3/15

Day 3: Strings in C#

1. What is a String?

- A string is a sequence of characters.
- In C#, string is a reference type and it is immutable (cannot be changed).
- We use double quotes (" ") to create a string.

2. Key Characteristics

- Immutable (creates a new string on change).
- Indexing starts from 0.
- Supports many built-in methods.
- Can be null or empty.

3. String Operations (Examples)

a) Create a string

```
string name = "Ashish";
Console.WriteLine(name);   // Ashish
```

b) Get length

```
string text = "Hello";
Console.WriteLine(text.Length);   // 5
```

c) Access a character

```
string text = "Hello";
Console.WriteLine(text[0]);   // H
```

d) Concatenate strings

```
string first = "Hello";
string last = "World";
string full = first + " " + last;
Console.WriteLine(full);   // Hello World
```

4. String Traversal

```
string text = "CSharp";
for (int i = 0; i < text.Length; i++)
{
    Console.WriteLine(text[i]);
}
```

It visits each character one by one.

Time: O(n)
Space: O(1)

5. Check for Palindrome

```
using System;
class Program
{
   static bool IsPalindrome(string s)
   {
      s = s.ToLower();
      int left = 0, right = s.Length-1;
      while (left < right)
      {
          if (s[left] != s[right])
             return false;
          left++;
          right--;
      }
      return true;
   }
}
```

Compares characters from start and end. If they match, it's a palindrome.

6. Common String Methods

Method	Use
Length	Gets the length of the string
ToLower() / ToUpper()	Changes case
Substring()	Gets a part of the string
Contains()	Checks if a string exists
IndexOf()	Finds the first occurrence
Replace()	Replaces a string
Split()	Splits into an array

7. Practice Questions

1. Write a C# program to reverse a string.
2. Check if a string is a palindrome.
3. Count the number of vowels in a string.
4. Remove spaces from a string.
5. Find the frequency of each character in a string.

8. What I Learned

- Strings are immutable in C#.
- We can access characters using index.
- String operations are useful in real-world problems.
- Two pointers help in problems like palindrome check.

Keep Going ☺

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

13 Oct 2026

Day 4/15

Day 4 : Two Pointers

1. What is Two Pointer?

- Two Pointers is a technique where we use two indexes/pointers to solve a problem in a single pass.
- It helps in reducing time complexity and avoids extra space.
- Commonly used with arrays and strings.

2. When to Use?

- Finding a pair with a given sum.
- Removing duplicates from a sorted array.
- Checking if a string is a palindrome.
- Reversing an array.
- Comparing elements from opposite ends.

3. Types of Two Pointers

a) Opposite Direction (left & right)

- Pointers start from opposite ends and move towards each other.
- Example: Pair sum, palindrome.

b) Same Direction (slow & fast)

- Both pointers move in the same direction.
- Example: Removing duplicates.

c) Fast and Slow

- Pointers move at different speeds.
- Example: Detecting a cycle in a linked list.

4. Example 1: Pair with Given Sum (Sorted Array)

Array: [1, 2, 4, 6, 8, 10] Sum = 12

Left = 0, Right = 5

0	1	2	3	4	5	6
1	2	4	6	6	8	10

left → index 0 | right → index 5

```
while (left < right)
{
    int sum = arr[left] + arr[right];
    if (sum == target) {
        Console.WriteLine("Pair found");
        break;
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
}
```

Time: O(n)
Space: O(1)

5. Example 2: Remove Duplicates (Sorted Array)

Array: [1, 1, 2, 2, 3, 4, 4]

0	1	2	3	4	5	6
1	1	2	2	3	4	4

Use one pointer to track the next unique element and another to scan the array.

```
if (arr.Length > 0)
{
    int uniqueIndex = 0;
    for (int i = 1; i < arr.Length; i++)
    {
        if (arr[i] != arr[uniqueIndex])
        {
            uniqueIndex++;
            arr[uniqueIndex] = arr[i];
        }
    }
}
```

Result:

1	2	3	4

(first 4 elements are unique)

Time: O(n)
Space: O(1)

6. Example 3: Check for Palindrome

A palindrome reads the same forward and backward.

Example: madam, level, racecar

```
static bool IsPalindrome(string s)
{
    int left = 0, right = s.Length-1;
    while (left < right)
    {
        if (s[left] != s[right])
            return false;
        left++;
        right--;
    }
    return true;
}
```

Usage:

```
Console.WriteLine(
  IsPalindrome("madam"));
// True

Console.WriteLine(
  IsPalindrome("hello"));
// False
```

Time: O(n)
Space: O(1)

7. Key Takeaways

- Two pointers can reduce unnecessary comparisons.
- Sorted arrays are especially useful for pair-sum problems.
- Many solutions run in O(n) time.
- In-place techniques can avoid extra memory.
- Always check the conditions to move each pointer.

8. Practice Questions

1. Reverse an array using two pointers.
2. Find a pair with a target sum in a sorted array.
3. Remove duplicates from a sorted array.
4. Check whether a string is a palindrome.
5. Move all zeros to the end of an array.

Keep Going :)

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

14 Oct 2026
Day 5/15

Day 5 : Sliding Window

1. What is Sliding Window?

- Sliding Window is a technique used to process a continuous range of elements in an array or string.
- Instead of recalculating everything for every new range, we update the window by adding new elements and removing elements that are no longer needed.
- It is commonly used for subarray and substring problems.

2. Types of Sliding Window

a) Fixed-size window

- The window always contains a fixed number of elements.
- Examples: 1) Maximum sum of subarray of size k
  2) Average of every subarray of size k

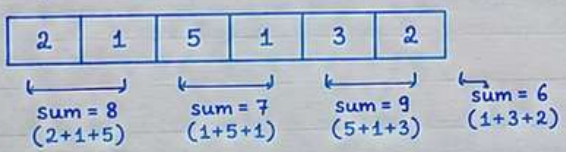
b) Variable-size window

- The window expands or shrinks depending on a condition.
- Examples: 1) Longest substring without repeating chars
  2) Smallest subarray with sum ≥ target (when all numbers are positive)

3. Example 1: Maximum Sum of Subarray of Size K

Given an array and a window size k, find the maximum sum of any contiguous subarray of size k.

Array: [2, 1, 5, 1, 3, 2] k = 3




```
int[] numbers = { 2, 1, 5, 1, 3, 2 };
int k = 3;
int windowSum = 0;
for (int i = 0; i < k; i++)
{
    windowSum += numbers[i];
}
int maxSum = windowSum;
for (int i = k; i < numbers.Length; i++)
{
    windowSum += numbers[i];
    windowSum -= numbers[i - k];
    maxSum = Math.Max(maxSum, windowSum);
}
Console.WriteLine(maxSum);
```

Output: 9

Time: O(n)
Space: O(1)

Explanation: The subarray [5, 1, 3] has the maximum sum of 9. Instead of calculating every window from scratch, we add the new element and subtract the element leaving the window.

4. Example 2: Maximum Sum with Fixed Window

For the array { 1, 4, 2, 10, 3, 1, 0, 20} and k = 3, the maximum sum is 21, from the subarray [1, 0, 20]. The sliding window technique avoids repeatedly summing all three elements for each window.

5. Example 3: Smallest Subarray with Sum ≥ Target

(Assuming all array elements are positive.)

```
static int MinSubArrayLength(int[] numbers, int target)
{
    int left = 0, sum = 0, minLength = int.MaxValue;
    for (int right = 0; right < numbers.Length, right++)
    {
        sum += numbers[right];
        while (sum >= target)
        {
            minLength = Math.Min(minLength, right - left+1);
            sum == numbers[left];
            left+=;
        }
    }
    return minLength == int.MaxValue ? 0 : minLength;
}
```

Example:

```
int[] numbers = { 2, 3, 1, 2, 4, 3 };
Console.WriteLine(MinSubArrayLength(numbers, 7));
```

Output: 2

Time: O(n)
Space: O(1)

Explanation: The smallest qualifying subarray is [4, 3]. This shrinking-window approach works only when all elements are positive. With negative numbers, the window sum may not behave predictably, so a different algorithm may be needed.

6. Sliding Window vs Brute Force

Approach	Time Complexity
Recalculate every fixed-size window	O(n x k)
Sliding window for fixed-size sums	O(n)
Variable-size window (positive values)	O(n)

The sliding window technique is useful when a problem involves contiguous elements and the window can be updated efficiently.

7. Key Takeaways

- What the sliding window technique is.
- Difference between fixed-size and variable-size windows.
- How to update a window without recalculating its entire sum.
- How sliding windows can reduce repeated work.
- Why problem constraints matter when choosing an algorithm.

8. Practice Questions

1. Find the maximum sum of a subarray of size k.
2. Calculate the average of every window of size k.
3. Find the smallest subarray with a sum ≥ target (positive numbers).
4. Find the longest substring without repeating characters.
5. Find the maximum number of consecutive ones after flipping at most one zero.

Keep Going :)

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

06 Oct 2026
Day 6/15

Day 6 : Hashing & Dictionary

1. What is Hashing?

- Hashing is a technique used to store and retrieve data efficiently using a hash function.
- It converts a key into a unique index (hash code) in a hash table.
- Helps in fast lookup, insertion and deletion of data.

2. Hash Function

- A hash function takes a key and returns an integer value (hash code).
- The hash code is used to determine the index in the hash table.
- Good hash functions minimize collisions.

3. Collision

- A collision occurs when two different keys produce the same hash code.
- To handle collisions, techniques like chaining or open addressing are used.

4. Hash Table

- A hash table is an array of buckets.
- Each bucket stores the key-value pairs.
- The key is hashed to find the bucket index.

Example (Simple Hash Function)

Key % size = index

Key	10	20	30	40	50
Hash (key % 5)	0	0	0	0	0

↳ All keys give same index (collision in this example).
In real use, a better hash function is used.

5. Dictionary in C#

- Dictionary<TKey, TValue> is a built-in collection in C# that uses hashing.
- It stores key-value pairs.
- Provides fast lookup, insertion and deletion.
- Keys must be unique.

Basic Syntax

```
Dictionary<int, string> dict = new Dictionary<int, string>();
dict[key] = value;
string value = dict[key];
bool exists = dict.ContainsKey(key);
```

Real World Uses

- Caches (e.g., browser cache)
- Database indexing
- User authentication
- Duplicate detection

Keep Going ☺

6. Example 1 : Using Dictionary

```
using System;
using System.Collections.Generic;

Dictionary<string, int> studentMarks = new Dictionary<string, int>();
studentMarks["Alice"] = 90;
studentMarks["Bob"] = 85;
studentMarks["Charlie"] = 92;

Console.WriteLine(studentMarks["Alice"]);   // 90
Console.WriteLine(studentMarks.ContainsKey("David")); // False
Console.WriteLine(studentMarks.Count);   // 3
```

Output

```
90
False
3
```

7. Example 2 : Frequency Count Using Dictionary

```
using System;
using System.Collections.Generic;

string text = "apple banana apple orange banana apple";
Dictionary<string, int> freq = new Dictionary<string, int>();

string[] words = text.Split(' ');
foreach (string word in words)
{
    if (freq.ContainsKey(word))
        freq[word]++;
    else
        freq[word] = 1;
}

foreach (var item in freq)
{
    Console.WriteLine($"{item.Key} : {item.Value}");
}
```

Output

```
apple : 3
banana : 2
orange : 1
```

8. Time Complexity

Operation	Time Complexity
Search (Dictionary)	O(1) average
Insert (Dictionary)	O(1) average
Delete (Dictionary)	O(1) average

9. Key Takeaways

- Hashing converts keys to hash codes.
- Helps in fast data retrieval.
- Collisions can occur and must be handled.
- Dictionary<TKey, TValue> is a useful collection in C#.
- Average time complexity for lookup, insertion and deletion is O(1).

10. Practice Questions

1. Implement a simple hash function.
2. Handle collisions using chaining.
3. Find the most frequent word in a sentence.
4. Use a Dictionary to check if two strings are anagrams.
5. Compare Dictionary vs List for searching elements.

15 Days of DSA with C#

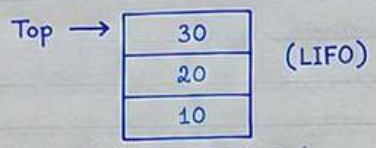
Better Code
Better Thinking
Better Interviews

16 Oct 2026
Day 7/15

Day 7 : Stack

1. What is Stack ?

- Stack is a linear data structure that follows the **LIFO (Last In, First Out)** principle.
- The last element added is the first one removed.
- Think of a stack of plates - you add and remove plates from the top.



Last In, First Out

2. Basic Stack Operations

Operation	Description
Push()	Adds an element to the top
Pop()	Removes and returns the top element
Peek()	Returns the top element without removing it
Count	Returns the number of elements
Clear()	Removes all elements

3. Example 1 : Basic Stack Operations

C# provides the built-in Stack<T> collection.

```
using System;
using System.Collections.Generic;

Stack<int> stack = new Stack<int>();
stack.Push(10);
stack.Push(20);
stack.Push(30);

Console.WriteLine(stack.Peek());    // 30
Console.WriteLine(stack.Pop());     // 30
Console.WriteLine(stack.Peek());    // 20
Console.WriteLine(stack.Count);     // 2
```

Output:

```
30
30
20
2
```

Explanation: The last element pushed was 30, so it is the first element returned by Peek() and Pop().

4. Example 2 : Reverse a String Using a Stack

```
using System;
using System.Collections.Generic;

string text = "HELLO";
Stack<char> stack = new Stack<char>();
foreach (char ch in text)
{
    stack.Push(ch);
}
while (stack.Count > 0)
{
    Console.Write(stack.Pop());
}
```

Output:

```
OLLEH
```

Time Complexity : O(n)
Space Complexity : O(n)

Explanation: Characters are pushed in their original order and popped in reverse order.

5. Example 3 : Check Balanced Parentheses

A stack can help determine whether brackets are properly matched and nested.

```
using System;
using System.Collections.Generic;
static bool IsBalanced(string expression)
{
    Stack<char> stack = new Stack<char>();
    foreach (char ch in expression)
    {
        if (ch == '(')
        {
            stack.Push(ch);
        }
        else if (ch == ')')
        {
            if (stack.Count == 0) return false;
            stack.Pop();
        }
    }
    return stack.Count == 0;
}
Console.WriteLine(IsBalanced("(a+b)"));  // True
Console.WriteLine(IsBalanced("(a+b"));   // False
Console.WriteLine(IsBalanced("a+b)"));   // False
```

Output:

```
True
False
False
```

Time Complexity : O(n)
Space Complexity : O(n)

Note: This example checks parentheses () only. Handling [] and {} requires checking matching bracket types as well.

6. Time Complexity

Operation	Time Complexity
Push()	O(1) amortized
Pop()	O(1)
Peek()	O(1)
Count	O(1)
Search for an element	O(n)

7. What I Learned

- The LIFO principle.
- Basic stack operations in C#.
- How to reverse a string using a stack.
- How stacks help solve bracket-matching problems.
- Common stack operation complexities.

8. Practice Questions

1. Reverse a string using a stack.
2. Check whether parentheses are balanced.
3. Implement a stack using an array.
4. Find the next greater element for each element in an array.
5. Design a stack that supports retrieving the minimum element efficiently.

9. 15 Days of DSA with C#

Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
Big O & Complexity ✓	Arrays ✓	Strings ✓	Two Pointers ✓	Sliding Window ✓	Hashing & Dictionary ✓	Stack ✓

Keep Going ☺

15 Days of DSA with C#

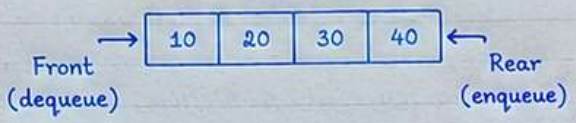
Better Code
Better Thinking
Better Interviews

17 Oct 2026
Day 8/15

Day 8: Queue

1. What is Queue?

- Queue is a linear data structure that follows the FIFO (First In, First Out) principle.
- The first element added is the first one removed.
- It works like a queue at a ticket counter.



2. Queue Operations

Operation	Description
Enqueue()	Adds an element to the end (rear).
Dequeue()	Removes and returns the front element.
Peek()	Returns the front element without removing it.
Count	Returns the number of elements.
Clear()	Removes all elements.

3. Example 1: Basic Queue Operations

C# provides the built-in Queue<T> collection.

```
using System;
using System.Collections.Generic;

Queue<int> queue = new Queue<int>();

queue.Enqueue(10);
queue.Enqueue(20);
queue.Enqueue(30);
queue.Enqueue(30);

Console.WriteLine(queue.Peek());      // 10
Console.WriteLine(queue.Dequeue());   // 10
Console.WriteLine(queue.Peek());      // 20
Console.WriteLine(queue.Count);       // 2
```

Output:

```
10
10
20
2
```

Explanation: The first element added was 10, so it is the first element returned by Peek() and Dequeue().

4. Example 2: Process Tasks in Order

```
using System;
using System.Collections.Generic;

Queue<string> tasks = new Queue<string>();

tasks.Enqueue("Send Email");
tasks.Enqueue("Generate Report");
tasks.Enqueue("Backup Data");

while (tasks.Count > 0)
{
    string task = tasks.Dequeue();
    Console.WriteLine($"Processing: {task}");
}
```

Output:

```
Processing: Send Email
Processing: Generate Report
Processing: Backup Data
```

Explanation: Tasks are processed in the same order in which they were added.

5. Example 3: Reverse a Queue

A stack can help reverse the order of elements in a queue.

```
using System;
using System.Collections.Generic;

Queue<int> queue = new Queue<int>();
queue.Enqueue(1);
queue.Enqueue(2);
queue.Enqueue(3);

Stack<int> stack = new Stack<int>();

while (queue.Count > 0)
{    stack.Push(queue.Dequeue());

}

while (stack.Count > 0)
{
    queue.Enqueue(stack.Pop());
}
Console.WriteLine(string.Join(", ", queue));
```

Output:

```
3, 2, 1
```

Time Complexity: O(n)
Auxiliary Space: O(n)

6. Time Complexity

Operation	Typical Time Complexity
Enqueue()	O(1) amortized
Dequeue()	O(1)
Peek()	O(1)
Count	O(1)
Search for an element	O(n)

7. Stack vs Queue

Feature	Stack	Queue
Principle	LIFO	FIFO
Insert operation	Push()	Enqueue()
Remove operation	Pop()	Dequeue()
Removal happens at	Top	Front
Example	Undo operation	Task scheduling

8. What I Learned

- The FIFO principle.
- Basic queue operations in C#.
- How queues process tasks in order.
- How a stack can be used to reverse a queue.
- Where queues are useful in real-world applications.

9. Practice Questions

1. Implement a queue using an array.
2. Reverse a queue using a stack.
3. Generate binary numbers from 1 to n using a queue.
4. Implement a circular queue.
5. Explain the difference between a stack and a queue.

10. 15 Days of DSA with C#

Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Day 8
Big O & Complexity	Arrays	Strings	Two Pointers	Sliding Window	Hashing & Dictionary	Stack	Queue
✓	✓	✓	✓	✓	✓	✓	✓

Keep Going ☺

15 Days of DSA with C#

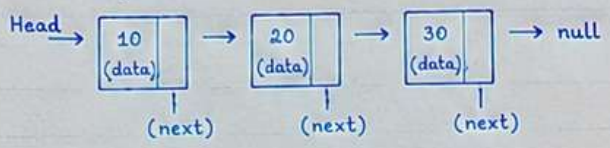
Better Code
Better Thinking
Better Interviews

18 Oct 2026
Day 9/15

Day 9 : Linked List

1. What is a Linked List ?

- A linked list is a linear data structure made up of nodes.
- Each node contains data and a reference to the next node.
- Unlike arrays, nodes are not stored in contiguous memory locations.



2. Types of Linked Lists

1. Singly Linked List — points to next node.
2. Doubly Linked List — points to next and previous node.
3. Circular Linked List — last node points to first node.

3. Node Class (C#)

```
class Node
{
    public int Data;
    public Node? Next;
    public Node(int data)
    {
        Data = data;
        Next = null;
    }
}
```

Node contains data and a reference to next node.

4. Example 1 : Create a Singly Linked List

```
Node first = new Node(10);
Node second = new Node(20);
Node third = new Node(30);
first.Next = second;
second.Next = third;

// Traversal
Node? current = first;
while (current != null)
{
    Console.Write(current.Data + " ");
    current = current.Next;
}
```

Output:

```
10 20 30
```

→ Traversal starts from the first node and continues until Next is null.

5. Example 2 : Insert at the Beginning

```
static Node InsertAtBeginning(Node? head, int data)
{
    Node newNode = new Node(data);
    newNode.Next = head;
    return newNode;
}
```

Time Complexity: O(1)

→ New node becomes the head of the list.

15 Days of DSA with C#

Day	2	3	4	5	6	7	8	9	10	11	12	13	14	15
✓	✓	✓	✓	✓	✓	✓	✓	✓						

6. Example 3 : Search for an Element

```
static bool Search(Node? head, int target)
{
    Node? current = head;
    while (current != null)
    {
        if (current.Data == target)
        {
            return true;
        }
        current = current.Next;
    }
    return false;
}
```

Output:

```
Search(head, 10) → True
Search(head, 50) → False
```

Time Complexity: O(n)
Space Complexity: O(1)

7. Example 4 : Delete a Node (by value)

```
static Node? Delete(Node? head, int target)
{
    if (head == null)
        return null;
    if (head.Data == target)
        return head.Next;
    Node current = head;
    while (current.Next != null)
    {
        if (current.Next.Data == target)
        {
            current.Next = current.Next.Next;
            break;
        }
        current = current.Next
    }
    return head;
}
```

Time Complexity: O(n)
Space Complexity: O(1)

8. Common Operations & Time Complexity (Singly Linked List)

Operation	Time Complexity
Access by position	O(n)
Search by value	O(n)
Insert at head	O(n)
Delete at head	O(1)
Insert after a known node	O(1)
Delete after a known node	O(1)

Note: Finding the node or its previous node may require O(n) traversal.

9. Array vs Linked List

Feature	Array	Linked List
Indexed access	O(1)	O(n)
Search	O(n)	O(n)
Insert at beginning	O(n)	O(1) (update head)
Memory layout	Contiguous	Non-contiguous
Extra memory	Low	Reference + data

10. Key Takeaways

- Nodes contain data and next reference.
- Linked lists are not stored in contiguous memory.
- Insertion and deletion (at head) can be O(1).
- Access and search are O(n).
- Linked lists are useful when frequent insertions/deletions are needed.

11. Practice Questions

1. Reverse a singly linked list.
2. Find the middle node of a linked list.
3. Detect a cycle in a linked list.
4. Find the nth node from the end.
5. Merge two sorted linked lists.

Keep Going ☺

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

15 Oct 2026
Day 10/15

Day 10 : Recursion

1. What is Recursion ?

- Recursion is a technique where a function calls itself to solve a problem.
- It breaks a problem into smaller subproblems of the same type.
- It is useful for problems that can be divided into smaller, similar parts.

2. How Recursion Works ?

1. Define the base case (when to stop).
2. Define the recursive case (call the function with a smaller problem).
3. The function keeps calling itself until the base case is reached.

3. Example : Factorial

$n! = n \times (n-1) \times (n-2) \times \dots \times 1$

```
Factorial(5) = 5 * 4 * 3 * 2 * 1
             = 120
```

```
Base case :
if (n == 0)
    return 1;
```

4. Factorial in C#

```
static int Factorial(int n)
{
    if (n == 0)
        return 1;

    return n * Factorial(n - 1);
}
```

Output :

```
Factorial(5)
= 120
```

5. Fibonacci Series

$F(n) = F(n-1) + F(n-2)$

Base cases : $F(0) = 0$, $F(1) = 1$

```
Sequence :
0, 1, 1, 2, 3, 5, 8, 13, ...
```

6. Fibonacci in C# (Recursive)

```
static int Fibonaccl(int n)
{
    if (n <= 1)
        return n;

    return Fibonacci(n - 1) + Fibonacci(n - 2);
}
```

Output :

```
Fibonacci(6)
= 8
```

7. Fibonacci Optimized (Memoization)

Store previously calculated results in a dictionary.

```
static int FibonacciMemo(int n, Dictionary<int, int> memo)
{
    if (n <= 1)
        return n;

    if (memo.TryGetValue(n, out int result))
        return result;

    result = FibonacciMemo(n - 1, memo) + FibonacciMemo(n - 2, memo);
    memo[n] = result;
    return result;
}
```

Output :

```
Fibonacci(6)
= 8
```

Time Complexity : O(n) | Space Complexity : O(n)

8. Fibonacci Using Tabulation (Bottom-Up)

Build the result using a table/array.

```
static int FibonacciTabulation(int n)
{
    if (n <= 1)
        return n;

    int[] dp = new int[n + 1];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++)
    {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Output :

```
Fibonacci(6)
= 8
```

Time Complexity : O(n) | Space Complexity : O(n)

9. Key Takeaways

- Recursion solves a problem by calling itself.
- A base case is necessary to stop the recursion.
- Each recursive call works on a smaller subproblem.
- Memoization (top-down) stores previous results.
- Tabulation (bottom-up) builds the result step by step.
- Some problems are easier to solve using recursion.
- Recursion can be less efficient without optimization.

10. Practice Questions

1. Find the factorial of a number using recursion.
2. Print the first n Fibonacci numbers using recursion.
3. Implement Fibonacci using memoization.
4. Implement Fibonacci using tabulation.
5. Compare the time and space complexity of all three approaches.

11. 15 Days of DSA with C#

- ☑ Day 1 : Big O & Complexity
- ☑ Day 2 : Arrays
- ☑ Day 3 : Strings
- ☑ Day 4 : Two Pointers
- ☑ Day 5 : Sliding Window
- ☑ Day 6 : Hashing & Dictionary
- ☑ Day 7 : Stack
- ☑ Day 8 : Queue
- ☑ Day 9 : Linked List
- ☑ Day 10 : Recursion
- ☐ Day 11 : Binary Search
- ☐ Day 12 : Sorting Algorithms
- ☐ Day 13 : Trees & Binary Trees
- ☐ Day 14 : Graph Basics
- ☐ Day 15 : Dynamic Programming + Final Review

Keep Going ☺

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

20 Oct 2026
Day 11/15

Day 11 : Binary Search

1. What is Binary Search ?

- Binary Search is an efficient searching algorithm used to find an element in a sorted array.
- It works by repeatedly dividing the search interval in half.
- It compares the middle element with the target value and reduces the search space.

2. When to use ?

- The data must be sorted (ascending or descending). Works best with large datasets.
- Not suitable for unsorted arrays or linked lists (without modification).

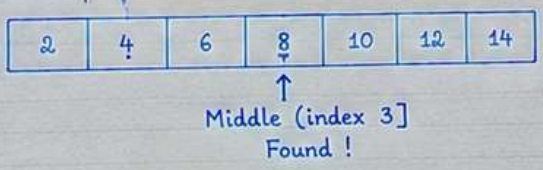
3. How it works ?

1. Find the middle element.
2. If it matches, return the index.
3. If the middle element is less than the target, search in the right half.
4. If it is greater, search in the left half.
5. Repeat until the element is found or the range ends.

4. Example

Array : [2, 4, 6, 8, 10, 12, 14] (sorted)

Search for : 8



5. Binary Search in C# (Iterative)

```
static int BinarySearch(int[] arr, int target)
{
    int left = 0;
    int right = arr.Length - 1;

    while (left <= right)
    {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target)
            return mid;

        if (arr[mid] < target)
            left = mid + 1;

        else
            right = mid - 1;
        }
    }

    return -1;   // not found
}
```

Output :

```
Index : 3
(for target 8)
```

Time Complexity : O(log n)

Space Complexity : O(1)

6. Binary Search in C# (Recursive)

```
static int BinarySearchRecursive(int[] arr, int target,
                                 int left, int right)
{
    if (left > right)
        return -1;

    int mid = left + (right - left) / 2;

    if (arr[mid] == target)
        return mid;

    if (arr[mid] < target)
        return BinarySearchRecursive(arr, target,
                                     mid + 1, right);
    else
        return BinarySearchRecursive(arr, target,
                                     left, mid - 1);
}
```

-- Call :

```
BinarySearchRecursive
(arr, 8, 0, arr.Length-1)
```

Time Complexity : O(log n)

Space Complexity : O(log n), (recursion stack)

7. Time & Space Complexity Summary

Approach	Time Complexity	Space Complexity
Iterative	O(log n)	O(1)
Recursive	O(log n)	O(log n)

8. Key Takeaways

- Binary Search works only on sorted data.
- It reduces the search space by half each time.
- Time complexity is O(log n), which is much faster than linear search (O(n)).
- Iterative approach uses constant space.
- Recursive approach uses O(log n) space due to call stack.
- The middle element is calculated using: mid = left + (right - left) / 2 (to avoid overflow).

9. Practice Questions

1. Implement iterative binary search in C#.
2. Implement recursive binary search in C#.
3. Modify the code to find the first occurrence of a number in a sorted array with duplicates.
4. Handle cases when the array is empty.
5. Test with different target values (first, middle, last, not found).

10. 15 Days of DSA with C# (Progress)

- ☑ Day 1 : Big O & Complexity
- ☑ Day 2 : Arrays
- ☑ Day 3 : Strings
- ☑ Day 4 : Two Pointers
- ☑ Day 5 : Sliding Window
- ☑ Day 6 : Hashing & Dictionary
- ☑ Day 7 : Stack
- ☑ Day 8 : Queue
- ☑ Day 9 : Linked List
- ☑ Day 10 : Recursion
- ☑ Day 11 : Binary Search
- ☐ Day 12 : Sorting Algorithms
- ☐ Day 13 : Trees & Binary Trees
- ☐ Day 14 : Graph Basics
- ☐ Day 15 : Dynamic Programming + Final Review

Day 11 completed ☺

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

21 Oct 2026
Day 12/15

Day 12 : Sorting Algorithms

1. What are Sorting Algorithms ?

- Sorting algorithms arrange elements in a specific order (ascending or descending).
- They are important because sorted data can make searching and processing easier.
- Common sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort (advanced).

2. Types of Sorting Algorithms

1. Bubble Sort — compare adjacent elements and swap.
2. Selection Sort — find the smallest element and place it.
3. Insertion Sort — insert element into the sorted part.

3. Bubble Sort in C#

```
static void BubbleSort(int[] arr)
{
    int n = arr.Length;
    for (int i = 0; i < n - 1; i++)
    {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

Output:

```
1, 2, 3, 5, 8
```

Time: $O(n^2)$
Space: $O(1)$

Idea: Repeatedly compare adjacent elements and swap if they are in the wrong order.

4. Selection Sort in C#

```
static void SelectionSort(int[] arr)
{
    int n = arr.Length;
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
```

Output:

```
1, 2, 3, 5, 8
```

Time: $O(n^2)$
Space: $O(1)$

Idea: Find the smallest element and place it in the correct position.

5. Insertion Sort in C#

```
static void InsertionSort(int[] arr)
{
    int n = arr.Length;
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Output:

```
1, 2, 3, 5, 8
```

Time: $O(n^2)$
Space: $O(1)$

Idea: Take each element and insert it into its correct position in the sorted section.

6. Time and Space Complexity

Algorithm	Best Case	Average Case	Worst Case	Extra Space
Bubble Sort (optimized)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Note: Best case $O(n)$ for Bubble Sort assumes the optimized version (checking if any swaps occurred).

7. Comparison

Algorithm	Main Idea	Useful For
Bubble Sort	Swap adjacent elements	Learning basics
Selection Sort	Select minimum element	Understanding selection logic
Insertion Sort	Insert into sorted section	Small or nearly sorted arrays

8. Key Takeaways

- Sorting arranges elements in a chosen order.
- Bubble Sort compares and swaps adjacent elements.
- Selection Sort repeatedly selects the smallest element.
- Insertion Sort inserts elements into their correct position.
- All three use $O(1)$ auxiliary space in the implementations.
- For large datasets, more efficient algorithms like Merge Sort and Quick Sort are preferred.

9. Practice Questions

1. Sort an array in ascending order using Bubble Sort.
2. Modify the sorting logic to arrange elements in descending order.
3. Implement Selection Sort and Insertion Sort.
4. Count comparisons and swaps for each algorithm.
5. Compare performance on an already-sorted and reverse-sorted array.

10. 15 Days of DSA with C#

- ☑ Day 1 : Big O & Complexity
- ☑ Day 2 : Arrays
- ☑ Day 3 : Strings
- ☑ Day 4 : Two Pointers
- ☑ Day 5 : Sliding Window
- ☑ Day 6 : Hashing & Dictionary
- ☑ Day 7 : Stack
- ☑ Day 8 : Queue
- ✓ Day 9 : Linked List
- ☑ Day 10 : Recursion
- ☑ Day 11 : Binary Search
- ☑ Day 12 : Sorting Algorithms
- ☑ Day 13 : Trees & Binary Trees
- ☑ Day 14 : Graph Basics
- ☑ Day 15 : Dynamic Programming + Final Review

Day 12 complete! ☺

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

22 Oct 2026
Day 13/15

Day 13 : Trees & Binary Trees

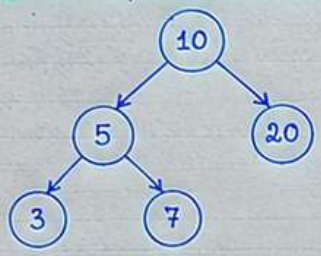
1. What are Trees & Binary Trees?

- Trees are a hierarchical data structure used to organize data.
- A binary tree is a tree in which each node can have at most two children (left and right).
- It consists of nodes and edges.

2. Important Terms

- Root : Topmost node of a tree.
- Parent : A node that has one or more children.
- Child : A node below another node.
- Leaf : A node with no children.
- Edge : Connection between two nodes.
- Height : Number of edges in the longest path from a node to a leaf.

3. Example of a Binary Tree



Root : 10
Children of 10 : 5, 20
Leaf nodes : 3, 7, 20

4. Creating a Binary Tree in C#

```
using System;
class Node
{
    public int Data;
    public Node? Left;
    public Node? Right;
    public Node(int data)
    {
        Data = data;
    }
}
class Program
{
    static void Main()
    {
        Node root = new Node(10);
        root.Left = new Node(5);
        root.Right = new Node(20);
        root.Left.Left = new Node(3);
        root.Left.Right = new Node(7);
        Console.WriteLine(root.Data);
    }
}
```

Output :

```
10
```

5. Tree Traversal

Traversal means visiting every node in a specific order.

a) Inorder (Left → Root → Right)

```
static void Inorder(Node? root)
{
    if (root == null) return;
    Inorder(root.Left);
    Console.Write(root.Data + " ");
    Inorder(root.Right);
}
```

Output :

```
3 5 7 10 20
```

b) Preorder (Root → Left → Right)

```
static void Preorder(Node? root)
{
    if (root == null) return;
    Console.Write(root.Data + " ");
    Preorder(root.Left);
    Preorder(root.Right);
}
```

Output :

```
10 5 3 7 20
```

c) Postorder (Left → Right → Root)

```
static void Postorder(Node? root)
{
    if (root == null) return;
    Postorder(root.Left);
    Postorder(root.Right);
    Console.Write(root.Data + " ");
}
```

Output :

```
3 7 5 20 10
```

6. Time and Space Complexity

Operation	Time Complexity
Visit all nodes (traversal)	O(n)
Search for a value	O(n)
Insert a node	Depends on approach

Auxiliary space for recursive traversal: O(h)

h = height of tree.

- Balnced tree : O(log n)
- Skewed tree : O(n)

n = number of nodes

7. Tree vs Binary Tree

Feature	General Tree	Binary Tree
Children per node	Can vary	At most two
Child positions	Depends on tree	Left and right
Example use	Hierarchical data	Searching & hierarchical processing

8. Key Takeaways

- Trees represent hierarchical relationships.
- Binary tree nodes have at most two children.
- Traversal helps visit every node in a specific order.
- Inorder, Preorder and Postorder use different sequences.
- Recursive methods are commonly used.
- A general binary tree is not automatically a BST.

9. Practice Questions

1. Create a binary tree with five nodes.
2. Implement inorder, preorder and postorder traversal.
3. Count the total number of nodes.
4. Find the maximum value in a tree.
5. Calculate the height of a tree.

10. 15 Days of DSA with C#

- ☑ Day 1 : Big O & Complexity
- ☑ Day 2 : Arrays
- ☑ Day 3 : Strings
- ☑ Day 4 : Two Pointers
- ☑ Day 5 : Sliding Window
- ☑ Day 6 : Hashing & Dictionary
- ☑ Day 7 : Stack
- ☑ Day 8 : Queue
- ☑ Day 9 : Linked List
- ☑ Day 10 : Recursion
- ☑ Day 11 : Binary Search
- ☑ Day 12 : Sorting Algorithms
- ☑ Day 13 : Trees & Binary Trees
- ☑ Day 14 : Graph Basics
- ☑ Day 15 : Dynamic Programming + Final Review

Day 13 completed. ☺

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

23 Oct 2026
Day 14/15

Day 14 : Graph Basics

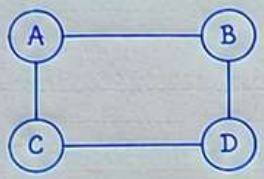
1. What is a Graph?

- A graph is a data structure used to represent connections and relationships between objects.
- It consists of vertices (nodes) and edges.
- Graphs are widely used in maps, social networks, computer networks and recommendation systems.

2. Types of Graphs

1. Undirected Graph — Edges have no direction.
2. Directed Graph — Edges have a direction (A → B).
3. Weighted Graph — Edges have values (distance, cost).
4. Unweighted Graph — Edges represent connections only.

3. Example of a Graph (Undirected)



Vertices : A, B, C, D
Edges : (A-B), (A-C), (B-D), (C-D)

4. Representing a Graph Using Adjacency List

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        Dictionary<string, List<string>> graph = new()
        {
            ["A"] = new List<string> { "B", "C" },
            ["B"] = new List<string> { "A", "D" },
            ["C"] = new List<string> { "A", "D" },
            ["D"] = new List<string> { "B", "C" }
        };

        foreach (var vertex in graph)
        {
            Console.WriteLine($"{vertex.Key}: {string.Join(", ", vertex.Value)}");
        }
    }
}
```

Output :

```
A: B, C
B: A, D
C: A, D
D: B, C
```

5. Breadth-First Search (BFS)

- Visits nodes level by level.
- Uses a queue to keep track of nodes.

```
static void BFS(Dictionary<string, List<string>> graph,
                string start)
{
    Queue<string> queue = new();
    HashSet<string> visited = new();

    queue.Enqueue(start);
    visited.Add(start);

    while (queue.Count > 0)
    {
        string current = queue.Dequeue();
        Console.Write(current + " ");

        foreach (string neighbor in graph[curment])
        {
            if (visited.Add(neighbor))
            {
                queue.Enqueue(neighbor);
            }
        }
    }
}
```

Call :

```
BFS(graph, "A");
```

Output :

```
A B C D
```

6. Depth-First Search (DFS)

- Explores one path as deeply as possible.
- Uses recursion or a stack.

```
static void DFS(Dictionary<string, List<string>> graph,
    string current, HashSet<string> visited)
{
    if (!visited.Add(current))
        return;

    Console.Write(current + " ");

    foreach (string neighbor in graph[current])
    {
        DFS(graph, neighbor, visited);
    }
}
```

Call :

```
DFS(graph, "A",
new HashSet<string>());
```

Output :

```
A B D C
```

7. Time and Space Complexity

Operation	Complexity
Store graph (adjacency list)	O(V + E) space
BFS traversal	O(V + E)
DFS traversal	O(V + E)
Check if edge exists (list)	O(degree of vertex)

V = number of vertices, E = number of edges

8. BFS vs DFS

Feature	BFS	DFS
Approach	Level by level	Deep first
Common implementation	Queue	Recursion / Stack
Useful for	Shortest path (in unweighted graph)	Exploring paths and connectivity
Traversal order	Depends on neighbor order	Depends on neighbor order

9. Real-World Applications

- Maps and route planning
- Social networks and friend connections
- Computer network routing
- Website link analysis
- Recommendation systems

10. Key Takeaways

- Graphs represent relationships between nodes.
- Vertices are connected using edges.
- Graphs can be directed, undirected, weighted or unweighted.
- An adjacency list stores each vertex's neighbors.
- BFS uses a queue, while DFS uses recursion or a stack.
- A visited set helps avoid repeated processing and handles cycles safely.

15 Days of DSA with C#

- ☑ Day 1 : Big O & Complexity
- ☑ Day 2 : Arrays
- ☑ Day 3 : Strings
- ☑ Day 4 : Two Pointers
- ☑ Day 5 : Sliding Window
- ☑ Day 6 : Hashing & Dictionary
- ☑ Day 7 : Stack
- ☑ Day 8 : Queue
- ☑ Day 9 : Linked List
- ☑ Day 10 : Recursion
- ☑ Day 11 : Binary Search
- ☑ Day 12 : Sorting Algorithms
- ☑ Day 13 : Trees & Binary Trees
- ☑ Day 14 : Graph Basics
- ☐ Day 15 : Dynamic Programming + Final Review

Day 14 completed. ☺

15 Days of DSA with C#

Better Code
Better Thinking
Better Interviews

24 Oct 2026
Day 15/15

Day 15 : Dynamic Programming + Final Review

1. What is Dynamic Programming?

- Dynamic Programming (DP) is a problem solving technique used to solve complex problems by breaking them into smaller subproblems and storing the results.
- It helps avoid repeated calculations and improves performance.

2. When to use DP?

- Overlapping Subproblems : Same subproblems are solved multiple times.
- Optimal Substructure : Solution to a larger problem can be built from solutions to smaller problems.

3. Types of DP Approaches

1. Memoization (Top-Down) : Use recursion + store results in a dictionary or array.
2. Tabulation (Bottom-Up) : Solve smaller problems first and build up using a table / array.

4. Fibonacci Sequence

The Fibonacci sequence starts with 0, 1, and each next number is the sum of the previous two.

4.1. Recursive Approach

```
static int FibonacciRecursive(int n)
{
    if (n <= 1)
        return n;

    return FibonacciRecursive(n - 1) +
           FibonacciRecursive(n - 2);
}
```

Time Complexity: $O(2^n)$ (approx)

Space Complexity: $O(n)$

4.2. Memoization (Top-Down)

```
static int FibonacciMemo(int n, Dictionary<int, int> memo)
{
    if (n <= 1)
        return n;

    if (memo.TryGetValue(n, out int result))
        return result;

    result = FibonacciMemo(n - 1, memo) +
             FibonacciMemo(n - 2, memo);
    memo[n] = result;
    return result;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

4.3. Tabulation (Bottom-Up)

```
static int FibonacciTabulation(int n)
{
    if (n <= 1)
        return n;

    int[] dp = new int[n + 1];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++)
    {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

5. Example Output (n = 6)

```
Fibonacci(6) = 8
(for all 3 approaches)
```

6. Comparing the Approaches

Approach	Time Complexity	Space Complexity
Recursive	$O(2^n)$ (approx)	$O(n)$
Memoization	$O(n)$	$O(n)$
Tabulation	$O(n)$	$O(n)$

7. Key Takeaways

- DP avoids repeated calculations by storing results.
- Memoization follows a top-down approach.
- Tabulation follows a bottom-up approach.
- Choose the right approach based on the problem and constraints.
- Understand time and space complexity to compare solutions.
- Not every problem needs DP – it is most useful when overlapping subproblems and optimal substructure exist.

8. Final Review – 15 Days of DSA with C#

- ☑ Day 1 : Big O & Complexity
- ☑ Day 2 : Arrays
- ☑ Day 3 : Strings
- ☑ Day 4 : Two Pointers
- ☑ Day 5 : Sliding Window
- ☑ Day 6 : Hashing & Dictionary
- ☑ Day 7 : Stack
- ☑ Day 8 : Queue
- ☑ Day 9 : Linked List
- ☑ Day 10 : Recursion
- ☑ Day 11 : Binary Search
- ☑ Day 12 : Sorting Algorithms
- ☑ Day 13 : Trees & Binary Trees
- ☑ Day 14 : Graph Basics
- ☑ Day 15 : Dynamic Programming + Final Review

9. What I Learned in This Journey

- Arrays and Strings
- Two Pointers and Sliding Window
- Hashing and Dictionaries
- Stack and Queue
- Linked Lists and Recursion
- Binary Search and Sorting
- Trees and Graphs
- Dynamic Programming basics
- Time and Space Complexity

10. What's Next?

- Solve more problems on each topic.
- Revisit concepts that need practice.
- Work on medium-level DSA questions using C#.

15 days completed –
but the learning continues! ☺